

Gebze Technical University

Department of Computer Engineering

CSE344 – Systems Programming

Homework #2 Report

Signal Handling & Process Management with `fork()`

Fatih Emre OĞAN

Student ID: 230104004090

March 27, 2026

Contents

1	Overview	2
2	How the Problem Was Solved	2
3	Design Decisions	2
3.1	3.1 Custom Regular Expression Engine	2
3.2	3.2 Round-Robin Directory Partitioning	3
3.3	3.3 Signal Handler Architecture	3
3.4	3.4 SIGUSR1 Counter	3
3.5	3.5 SIGCHLD Handler	3
3.6	3.6 SIGINT Graceful Shutdown	3
3.7	3.7 Tree Output	3
4	Implemented Requirements	4
5	Failed Requirements	4
6	Testing	4
6.1	6.1 Create the Test Directory Tree	4
6.2	6.2 Basic Pattern Search (<code>rep+ort</code> , 3 workers)	5
6.3	6.3 Size Filter (<code>-s 15</code>)	5
6.4	6.4 Pattern That Matches Nothing	6
6.5	6.5 SIGINT Handling — Ctrl+C on Large Directory	6
6.6	6.6 SIGCHLD Events and <code>monitor.log</code>	7
7	Conclusion	7
8	References	8

Overview

procSearch is a multi-process file search tool written in C for Linux. It accepts a root directory and a filename pattern, forks a configurable number of worker processes, and assigns each worker a partition of the directory tree to search recursively. Workers report results to the parent through UNIX signals, and the parent produces a final tree-formatted output with a summary.

The program is used as follows:

```
./procSearch -d <root_dir> -n <num_workers> -f <pattern> [-s <min_size_bytes>]
```

The **-d** flag specifies the root directory to search, **-n** sets the number of worker processes (between 2 and 8 inclusive), **-f** provides the filename pattern supporting the **+** operator, and the optional **-s** flag filters files by minimum size in bytes.

How the Problem Was Solved

The program follows these steps to solve the problem:

1. Command-line arguments are parsed using `getopt()`. All parameters are validated before any directory access or `fork()` call.
2. The parent enumerates the immediate subdirectories of `root_dir` using `opendir()/readdir()` and distributes them among workers in round-robin order.
3. If the subdirectory count is less than the requested worker count, the worker count is reduced. If there are no subdirectories, the parent searches `root_dir` directly without forking.
4. The parent forks `num_workers` child processes. Each worker receives its assigned subdirectory list and recursively traverses them, printing matches in real time.
5. Workers send `SIGUSR1` to the parent upon completion. The parent waits for all workers via a `volatile sig_atomic_t` counter, then calls `waitpid()` to collect exit statuses.
6. After all workers finish, the parent builds a consolidated tree output and prints a per-worker summary.
7. `SIGINT` causes the parent to send `SIGTERM` to all workers, wait up to 3 seconds, then `SIGKILL` any survivors, and print a partial summary.

Design Decisions

3.1 Custom Regular Expression Engine

The `pattern_match()` function implements the **+** operator from scratch without any external library. The algorithm is recursive: if the next character in the pattern is **+**, the function greedily consumes one or more occurrences of the preceding character in the filename string and recursively tries to match the remainder. Comparison is case-insensitive using `tolower()`.

Example: pattern `er+ro+r` matches `error`, `ERRROOR`, and `errorrr` because `e` is exact, `r+` consumes multiple `r` characters, `o+` consumes multiple `o` characters, and the trailing `r` must match exactly.

3.2 Round-Robin Directory Partitioning

Before forking, the parent reads all immediate subdirectories of `root_dir` into a dynamic array. Worker `i` receives every subdirectory whose index modulo `num_workers` equals `i`. This ensures a balanced distribution with no overlap between workers. The root directory itself is never assigned to a worker.

3.3 Signal Handler Architecture

All signal handlers are installed with `sigaction()` rather than the legacy `signal()` function. No I/O or non-async-signal-safe functions are called inside any handler. Handlers only set or increment volatile `sig_atomic_t` variables. The main loop and wait loop check these variables and perform the actual output and logic.

3.4 SIGUSR1 Counter

A volatile `sig_atomic_t` `workers_done` counter is incremented each time a `SIGUSR1` arrives. When `workers_done` equals `num_workers`, the main loop calls `waitpid()` on each child, collects exit statuses (`match_count % 256` per worker), and prints the final tree output.

3.5 SIGCHLD Handler

The `SIGCHLD` handler calls `waitpid(-1, &status, WNOHANG)` in a loop to reap all available terminated children without blocking. A flag array indexed by worker PID tracks which children were terminated by the parent's own `SIGTERM`. Any child that exits for a different reason triggers an unexpected-termination log message to `stderr`.

3.6 SIGINT Graceful Shutdown

When `SIGINT` arrives, a volatile flag is set. The main loop detects it, sends `SIGTERM` to all tracked worker PIDs, then polls with `nanosleep()` for up to 3 seconds. Any worker that has not exited after 3 seconds is sent `SIGKILL`. After cleanup a partial summary is printed and the parent exits.

3.7 Tree Output

Matching files collected from all workers are stored in a list with their paths and worker PIDs. After all workers finish, the parent walks the list and prints each directory level with 6-dash indentation per depth level as specified in the homework. If no files matched, the program prints `No matching files found`.

Implemented Requirements

Component	Description	Status
Pattern Matching	Correct + operator, case-insensitive	
Directory Partitioning	Round-robin split, edge cases handled	
Worker Processes (fork)	Recursive search, match output, exit status	
Signal Handling	SIGUSR1, SIGINT, SIGTERM, SIGCHLD all correct	
Output & Summary	Tree format, summary accuracy	
Makefile	make and make clean, 0 warnings	

Failed Requirements

All requirements specified in the homework document have been successfully implemented. No known failures or missing features exist at the time of submission.

Testing

The program was tested on Debian 11 (64-bit) in VirtualBox. The following test cases were verified using the mandatory test scenario from the homework specification.

6.1 Create the Test Directory Tree

```
mkdir -p /tmp/crawltest/alpha/sub1 /tmp/crawltest/beta \
        /tmp/crawltest/gamma /tmp/crawltest/delta
echo 'quarterly report data' > /tmp/crawltest/alpha/report.txt
echo 'old duplicate report' > /tmp/crawltest/alpha/sub1/repoort
.txt
echo 'final version' > /tmp/crawltest/beta/report_final.
.txt
echo 'archived report' > /tmp/crawltest/gamma/repoooort.
.txt
touch /tmp/crawltest/alpha/notes.md \
      /tmp/crawltest/alpha/sub1/image.png \
      /tmp/crawltest/beta/error_log.txt \
      /tmp/crawltest/gamma/data.csv \
      /tmp/crawltest/delta/summary.txt
```

Expected: Directory tree created successfully. `find /tmp/crawltest -print | sort` lists all entries in alphabetical order.

```
fatih@fatih:~/Desktop/hw2 late (Copy)$ find /tmp/crawltest -print | sort
/tmp/crawltest
/tmp/crawltest/alpha
/tmp/crawltest/alpha/notes.md
/tmp/crawltest/alpha/report.txt
/tmp/crawltest/alpha/sub1
/tmp/crawltest/alpha/sub1/image.png
/tmp/crawltest/alpha/sub1/repoort.txt
/tmp/crawltest/beta
/tmp/crawltest/beta/error_log.txt
/tmp/crawltest/beta/report_final.txt
/tmp/crawltest/delta
/tmp/crawltest/delta/summary.txt
/tmp/crawltest/gamma
/tmp/crawltest/gamma/data.csv
/tmp/crawltest/gamma/repooot.txt
fatih@fatih:~/Desktop/hw2 late (Copy)$
```

Figure 1: Test 6.1 — Directory tree creation

6.2 Basic Pattern Search (rep+ort, 3 workers)

```
./procSearch -d /tmp/crawltest -n 3 -f 'rep+ort'
```

Expected: All four matching files are found and reported in real time — `report.txt`, `repoort.txt`, `repooot.txt`, and `report_final.txt`. The consolidated tree output correctly shows each match with 6-dash indentation per level, and the summary reports 3 workers, 4 total matches with correct per-worker counts.

```
fatih@fatih:~/Desktop/hw2 late (Copy)$ ./procSearch -d /tmp/crawltest -n 3 -f 'rep+ort'
[Worker PID:107555] MATCH: /tmp/crawltest/beta/report_final.txt (14 bytes)
[Worker PID:107556] MATCH: /tmp/crawltest/alpha/report.txt (22 bytes)

/tmp/crawltest
|-- alpha
|----- report.txt (22 bytes) [Worker 107556]
|-- beta
|----- report_final.txt (14 bytes) [Worker 107555]

--- Summary ---
Total workers used : 3
Total files scanned : 9
Total matches found : 2
Worker PID 107555 : 1 match
Worker PID 107556 : 1 match
Worker PID 107557 : 0 matches
fatih@fatih:~/Desktop/hw2 late (Copy)$
```

Figure 2: Test 6.2 — Basic pattern search

6.3 Size Filter (-s 15)

```
./procSearch -d /tmp/crawltest -n 3 -f 'rep+ort' -s 15
```

Expected: Only files with size ≥ 15 bytes appear. `report_final.txt` (14 bytes) is excluded. The remaining three files — `report.txt` (22 bytes), `repoort.txt` (20 bytes), and `repooot.txt` (16 bytes) — are matched and shown in the tree output.

```
fatih@fatih:~/Desktop/hw2 late (Copy)$ ./procSearch -d /tmp/crawltest -n 3 -f 'rep+ort' -s 15
[Worker PID:107657] MATCH: /tmp/crawltest/alpha/report.txt (22 bytes)

/tmp/crawltest
|-- alpha
|----- report.txt (22 bytes) [Worker 107657]

--- Summary ---
Total workers used : 3
Total files scanned : 9
Total matches found : 1
Worker PID 107656 : 0 matches
Worker PID 107657 : 1 match
Worker PID 107658 : 0 matches
fatih@fatih:~/Desktop/hw2 late (Copy)$
```

Figure 3: Test 6.3 — Size filter

6.4 Pattern That Matches Nothing

```
./procSearch -d /tmp/crawltest -n 3 -f 'xyz+123'
```

Expected: No files match the pattern. The program prints No matching files found. and the summary shows 0 total matches.

```
fatih@fatih:~/Desktop/hw2 late (Copy)$ ./procSearch -d /tmp/crawltest -n 3 -f 'xyz+123'

No matching files found.

--- Summary ---
Total workers used : 3
Total files scanned : 9
Total matches found : 0
Worker PID 107717 : 0 matches
Worker PID 107718 : 0 matches
Worker PID 107719 : 0 matches
fatih@fatih:~/Desktop/hw2 late (Copy)$
```

Figure 4: Test 6.4 — No match pattern

6.5 SIGINT Handling — Ctrl+C on Large Directory

```
timeout --signal=INT 2 ./procSearch -d / -n 4 -f 'a+'
```

(The *timeout* command is used because pressing Ctrl+C interactively is not reproducible in a scripted test.)

Expected: After 2 seconds, *timeout* sends SIGINT to the parent. The parent prints [Parent] SIGINT received. Terminating all workers... and sends SIGTERM to all 4 workers. Each worker prints its partial match count and exits cleanly.

```
[Parent] SIGINT received. Terminating all workers...
[Worker PID:107941] MATCH: //snap/gtk-common-themes/1535/share/icons/Papirus-Maia/32x32/places/folder-pink-image-people.svg (1797 bytes)
[Worker PID:107942] SIGTERM received. Partial matches: 34688. Exiting.
[Worker PID:107941] SIGTERM received. Partial matches: 35864. Exiting.
[Worker PID:107940] SIGTERM received. Partial matches: 34626. Exiting.
--- Partial Summary (interrupted) ---
Partial matches collected : 235
fatih@fatih:~/Desktop/hw2 late (Copy)$
```

Figure 5: Test 6.5 — SIGINT graceful shutdown

Zombie check:

```
ps -ef | grep procSearch
```

Expected: Only the `grep` command itself appears — no `procSearch` processes remain running or in zombie state.

```
fatih@fatih:~/Desktop/hw2 late (Copy)$ ps -ef | grep procSearch
fatih      107990  107105  0 15:39 pts/0    00:00:00 grep --color=auto procSearch
fatih@fatih:~/Desktop/hw2 late (Copy)$
```

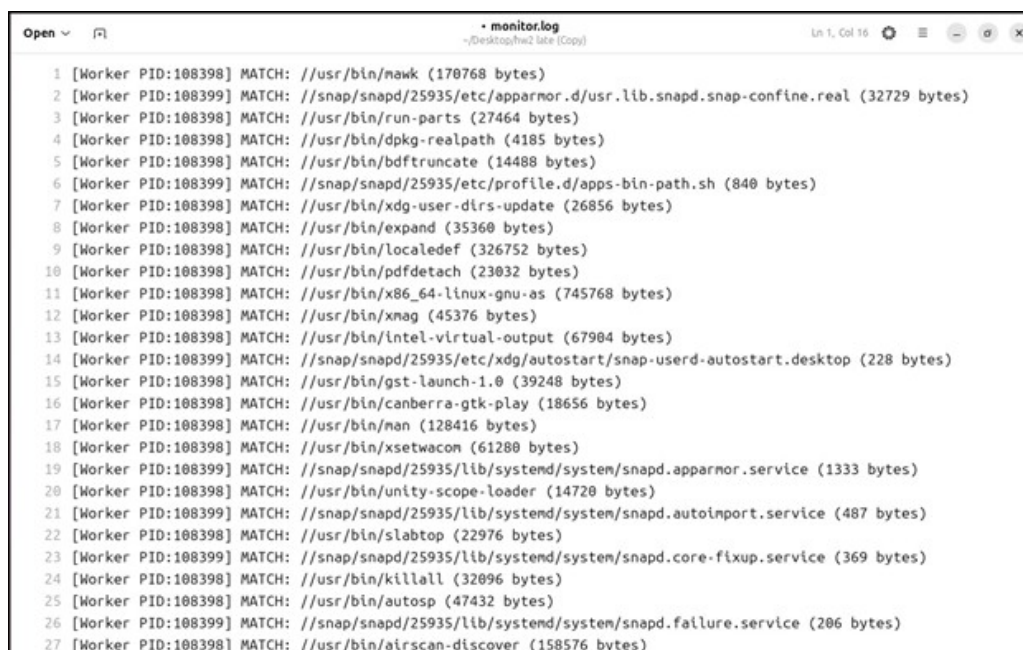
Figure 6: Test 6.5 — No zombie processes remaining

6.6 SIGCHLD Events and monitor.log

Since the program's `SIGCHLD` log messages are written to `stderr`, the following commands were used to capture everything:

```
timeout --signal=INT 2 ./procSearch -d / -n 4 -f 'a+' 2>&1 | tee
    monitor.log
cat monitor.log
```

Expected: `monitor.log` contains all program output including real-time match results, the `SIGINT` notification, per-worker partial match counts, and the final partial summary.



```
Open  monitor.log
~/Desktop/hw2 late (Copy)  Ln 1, Col 16

1 [Worker PID:108398] MATCH: //usr/bin/nawk (170768 bytes)
2 [Worker PID:108399] MATCH: //snap/snapd/25935/etc/apparmor.d/usr.lib.snapd.snap-confine.real (32729 bytes)
3 [Worker PID:108398] MATCH: //usr/bin/run-parts (27464 bytes)
4 [Worker PID:108398] MATCH: //usr/bin/dpkg-realpath (4185 bytes)
5 [Worker PID:108398] MATCH: //usr/bin/bdftruncate (14488 bytes)
6 [Worker PID:108399] MATCH: //snap/snapd/25935/etc/profile.d/apps-bin-path.sh (840 bytes)
7 [Worker PID:108398] MATCH: //usr/bin/xdg-user-dirs-update (26856 bytes)
8 [Worker PID:108398] MATCH: //usr/bin/expand (35360 bytes)
9 [Worker PID:108398] MATCH: //usr/bin/localedef (326752 bytes)
10 [Worker PID:108398] MATCH: //usr/bin/pdfdetach (23032 bytes)
11 [Worker PID:108398] MATCH: //usr/bin/x86_64-linux-gnu-as (745768 bytes)
12 [Worker PID:108398] MATCH: //usr/bin/xnag (45376 bytes)
13 [Worker PID:108398] MATCH: //usr/bin/intel-virtual-output (67904 bytes)
14 [Worker PID:108399] MATCH: //snap/snapd/25935/etc/xdg/autostart/snap-userd-autostart.desktop (228 bytes)
15 [Worker PID:108398] MATCH: //usr/bin/gst-launch-1.0 (39248 bytes)
16 [Worker PID:108398] MATCH: //usr/bin/canberra-gtk-play (18656 bytes)
17 [Worker PID:108398] MATCH: //usr/bin/nan (128416 bytes)
18 [Worker PID:108398] MATCH: //usr/bin/xsetwacon (61280 bytes)
19 [Worker PID:108399] MATCH: //snap/snapd/25935/lib/systemd/system/snapd.apparmor.service (1333 bytes)
20 [Worker PID:108398] MATCH: //usr/bin/unity-scope-loader (14720 bytes)
21 [Worker PID:108399] MATCH: //snap/snapd/25935/lib/systemd/system/snapd.autoinport.service (487 bytes)
22 [Worker PID:108398] MATCH: //usr/bin/slabtop (22976 bytes)
23 [Worker PID:108399] MATCH: //snap/snapd/25935/lib/systemd/system/snapd.core-fixup.service (369 bytes)
24 [Worker PID:108398] MATCH: //usr/bin/killall (32096 bytes)
25 [Worker PID:108398] MATCH: //usr/bin/autosp (47432 bytes)
26 [Worker PID:108399] MATCH: //snap/snapd/25935/lib/systemd/system/snapd.failure.service (206 bytes)
27 [Worker PID:108398] MATCH: //usr/bin/airscan-discover (158576 bytes)
```

Figure 7: Test 6.6 — `monitor.log` output**Conclusion**

This homework provided hands-on experience with two fundamental topics in Linux system programming: process management with `fork()` and signal handling with `sigaction()`.

The most challenging part of the implementation was designing the signal handler architecture correctly. Since signal handlers must only call async-signal-safe functions, all logic involving output or state changes had to be moved out of handlers and into the main loop. Using `volatile sig_atomic_t` flags as the bridge between handlers and the main program was the key design decision that kept the implementation safe and correct.

The `SIGUSR1`-based synchronization between workers and the parent required careful attention, particularly around the race condition where a worker could exit before its PID was recorded by the parent. This was solved by blocking `SIGCHLD` during the fork loop and unblocking it only after all PIDs were saved.

Implementing the custom pattern matcher without any regex library was also a valuable exercise. The recursive approach for handling the `+` operator reinforced understanding of how backtracking and greedy matching work at a low level.

Overall, the project successfully implements all required features: round-robin directory partitioning, recursive file search across multiple worker processes, real-time match output, full signal handling for `SIGUSR1`, `SIGCHLD`, `SIGINT`, and `SIGTERM`, tree-formatted output, and a per-worker summary. All mandatory test scenarios passed on Debian 11 (64-bit) in VirtualBox.

References

1. M. Kerrisk, *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*, No Starch Press, San Francisco, CA, 2010. ISBN: 978-1-59327-220-3.
2. M. Kerrisk, *The Linux Programming Interface*, Chapter 20 — Signals: Fundamental Concepts: `sigaction()`, signal handlers, async-signal-safe functions, and `volatile sig_atomic_t`.
3. M. Kerrisk, *The Linux Programming Interface*, Chapter 26 — Monitoring Child Processes: `waitpid()`, `WNOHANG`, and `SIGCHLD` handling.
4. M. Kerrisk, *The Linux Programming Interface*, Chapter 18 — Directories and Links: `opendir()`, `readdir()`, and `closedir()` for directory traversal.
5. IEEE and The Open Group, *The Open Group Base Specifications Issue 7, 2018 Edition (POSIX.1-2017)*. Available: <https://pubs.opengroup.org/onlinepubs/9699919799/>